

FULLY HOMOMORPHIC ENCRYPTION

ADVANCED TOPICS IN ~~CYBERSECURITY~~ CRYPTOGRAPHY (7CCSMATC)

Martin R. Albrecht

OUTLINE

Fully Homomorphic Encryption

Preliminaries

GSW

Applications



Vinod Vaikuntanathan. **CS 294-168 Lattices, Learning with Errors and Post-Quantum Cryptography.**

<https://people.csail.mit.edu/vinodv/CS294/>.

May 2020

FULLY HOMOMORPHIC ENCRYPTION

(FULLY HOMOMORPHIC) ENCRYPTION

- Encryption: used to protect data at rest or in transit
- Fully Homomorphic Encryption: supports arbitrary computations on encrypted data

- Concept originally proposed in [RAD78]
- Gentry's breakthrough [Gen09]
 - First candidate solution
 - Introduced the “bootstrapping” technique
- Much subsequent work
 - Basing security on standard (lattice) assumptions [BV11; Bra12; AP14; GSW13; BV14]
 - Efficiency improvements [GHS12; BGH13; AP13; AP14; DM15; CGGI20; CKKS17]
 - Implementations: HELib, SEAL, PALISADE, FHEW, TFHE, HeaAn, NTLlib

DEFINITION

$\text{KeyGen}(1^\lambda)$ produces a secret key sk , possibly together with a public evaluation key ek .

$\text{Enc}(\text{sk}, \mu)$ produces a ciphertext c where $\mu \in \{0, 1\}$.

$\text{Dec}(\text{sk}, c)$ outputs μ or $\perp$.

$\text{Eval}(\text{ek}, f, c_1, \dots, c_k)$ takes as input a $\text{poly}(\lambda)$ -size circuit that computes a function $f : \{0, 1\}^k \rightarrow \{0, 1\}$, as well as k ciphertexts $c_1, \dots, c_k$, and outputs a ciphertext c_f .

Correctness

$$\text{Dec}(\text{sk}, \text{Eval}(\text{ek}, f, \text{Enc}(\text{sk}, \mu_1), \dots, \text{Enc}(\text{sk}, \mu_k))) = f(\mu_1, \dots, \mu_k)$$

for all $f, \mu_1, \dots, \mu_k$ with probability $\geq 1 - \text{negl}(\lambda)$ over the sk, ek and the randomness of all the algorithms.

$|c_f| = \text{poly}(\lambda)$, independent of the circuit size of f .¹

¹Weaker compactness conditions are possible, and indeed, we will see one in the sequel.

$$(\text{Enc}(\text{sk}, \mu_1), \dots, \text{Enc}(\text{sk}, \mu_{p(\lambda)})) \approx_c (\text{Enc}(\text{sk}, \mu'_1), \dots, \text{Enc}(\text{sk}, \mu'_{p(\lambda)})) \ .$$

CHALLENGE

Show that no FHE scheme can be IND-CCA secure.

PRELIMINARIES

EIGENVALUES AND EIGENVECTORS

- Let $\mathbf{M}$ be a square matrix
- A non-zero vector $\mathbf{v}$ is an **eigenvector** of $\mathbf{M}$ if for some scalar λ (the **eigenvalue**):

$$\mathbf{v}^T \cdot \mathbf{M} = \lambda \cdot \mathbf{v}^T$$

- Eigenvalues capture how $\mathbf{M}$ scales its eigenvectors

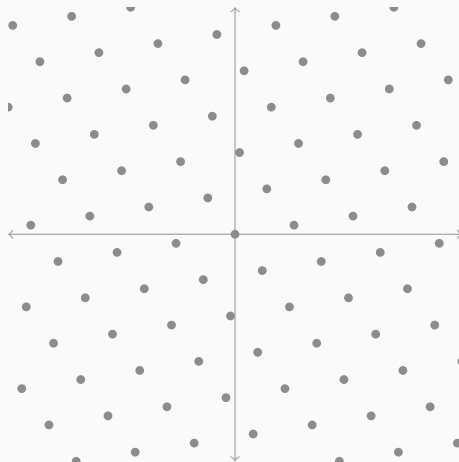
LATTICES

- A lattice is a discrete subgroup of $\mathbb{R}^d$
- It can be written as

$$\Lambda = \left\{ \sum_{i=0}^{d-1} v_i \cdot \mathbf{b}_i \mid v_i \in \mathbb{Z} \right\}$$

for some basis vectors $\mathbf{b}_i$.

- We write $\Lambda(\mathbf{B})$ for the lattices spanned by the columns of $\mathbf{B}$.
- A lattice is q -ary if it contains $q\mathbb{Z}^d$, e.g. $\{\mathbf{x} \in \mathbb{Z}^d \mid \mathbf{x} \cdot \mathbf{A} \equiv \mathbf{0}\}$ for some $\mathbf{A} \in \mathbb{Z}^{d \times d'}$.

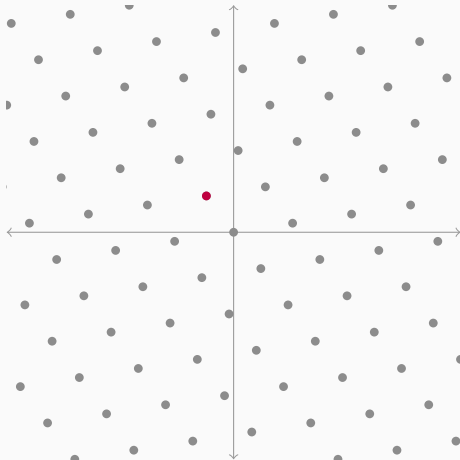


SHORTEST VECTOR PROBLEM

Definition

Given a lattice basis $\mathbf{B}$, find a shortest non-zero vector in $\Lambda(\mathbf{B})$.

- The most natural problem on lattices
- We write $\lambda_1(\Lambda)$ for the Euclidean norm of a shortest vector.
- NP-hard to solve exactly
- Cryptography relies on approximate variants without such a reduction



THE LEARNING WITH ERRORS PROBLEM (LWE)

Definition (LWE)

Let n, q be positive integers, χ be a probability distribution on $\mathbb{Z}$ and $\mathbf{s}$ be a uniformly random vector in $\mathbb{Z}_q^n$. We denote by $\mathcal{L}_{\mathbf{s}, \chi}$ the probability distribution on $\mathbb{Z}_q^n \times \mathbb{Z}_q$ obtained by choosing $\mathbf{a} \in \mathbb{Z}_q^n$ uniformly at random, choosing $e \in \mathbb{Z}$ according to χ and considering it in $\mathbb{Z}_q$, and returning $(\mathbf{a}, c) = (\mathbf{a}, \langle \mathbf{a}, \mathbf{s} \rangle + e) \in \mathbb{Z}_q^n \times \mathbb{Z}_q$.

- **Decision-LWE:** is the problem of deciding whether pairs $(\mathbf{a}, c) \in \mathbb{Z}_q^n \times \mathbb{Z}_q$ are sampled according to $\mathcal{L}_{\mathbf{s}, \chi}$ or the uniform distribution on $\mathbb{Z}_q^n \times \mathbb{Z}_q$.
- **Search-LWE:** is the problem of recovering $\mathbf{s}$ from pairs $(\mathbf{a}, c) = (\mathbf{a}, \langle \mathbf{a}, \mathbf{s} \rangle + e) \in \mathbb{Z}_q^n \times \mathbb{Z}_q$ sampled according to $\mathcal{L}_{\mathbf{s}, \chi}$.

BIT DECOMPOSITION

- We'll make use of an operation `bit_decomp` which splits a vector of integers $\leq q$ into a $\log q$ longer vector of the bits of the integers.
- For example, $(7, 2) \bmod 8$ becomes $(1, 1, 1, 0, 1, 0)$ which has length $2 \cdot \log 8 = 2 \cdot 3 = 6$.

```
def bit_decomp(v):  
    R = v.base_ring()  
    k = ZZ((R.order()-1).nbits())  
    w = vector(R, len(v)*k)  
    for i in range(len(v)):  
        for j in range(k):  
            if 2**j & ZZ(v[i]):  
                w[k*i+j] = 1  
            else:  
                w[k*i+j] = 0  
    return w
```

BIT COMPOSITION

- We also need a function which reverses the process, i.e. adds up the appropriate powers of two: $(1 + 2 + 4 = 7, 2)$
- The definition of `bit_comp` is a bit more general than just adding up bits.
- It will add up any integer entry of `v` multiplied with the appropriate power of two multiplied in.
- This is relevant for `flatten` which we define as `bit_decomp(bit_comp())`.

```
def bit_comp(v):
    R = v.base_ring()
    k = ZZ((R.order()-1).nbits())
    assert(k.divides(len(v)))
    w = vector(R, len(v)//k)
    for i in range(len(v)//k):
        for j in range(k):
            w[i] += 2**j * ZZ(v[k*i+j])
    return w

flatten = lambda v: bit_decomp(bit_comp(v))
```

POWERS OF 2

- We also define `powers_of_two` which produces a new vector from `v` as $(v_1, 2 \cdot v_1, 2^2 \cdot v_1, 2^{\lceil \log q \rceil} \cdot v_1, \dots, 2^{\lceil \log q \rceil} \cdot v_n)$
- For example the output of `powers_of_two` on `(3, 1)` for $q = 8$ is `(3, 6, 4, 1, 2, 4)`.

```
def powers_of_two(v):  
    R = v.base_ring()  
    k = ZZ((R.order()-1).nbits())  
    w = vector(R, len(v)*k)  
    for i in range(len(v)):  
        for j in range(k):  
            w[k*i+j] = 2**j * v[i]  
    return w
```

$$\langle \text{bit_decomp}(v), \text{powers_of_two}(w) \rangle = \langle v, w \rangle \quad \text{I}$$

$$\langle \text{bit_decomp}(v), \text{powers_of_two}(w) \rangle = \langle v, w \rangle$$

Example

$$5 \cdot 3 = (1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0) \cdot 3 = 1 \cdot (2^2 \cdot 3) + 0 \cdot (2^1 \cdot 3) + 1 \cdot (2^0 \cdot 3) = 15 \quad .$$

Implication $v := \text{bit_comp}(a)$

$$\langle \text{flatten}(a), \text{powers_of_two}(b) \rangle = \langle \text{bit_comp}(a), b \rangle$$

EXAMPLES $\langle \text{bit_decomp}(v), \text{powers_of_two}(w) \rangle = \langle v, w \rangle$

$$\langle (7, 2), (3, 1) \rangle = 23 \quad \text{and} \quad \langle (1, 1, 1, 0, 1, 0), (3, 6, 12, 1, 2, 4) \rangle = 3 + 6 + 12 + 2 = 23.$$

```
q = 8
R = IntegerModRing(q)
v = random_vector(R, 10)
w = random_vector(R, 10)
v.dot_product(w) == bit_decomp(v).dot_product(powers_of_two(w))
```

$$\langle \mathbf{a}, \text{powers_of_two}(\mathbf{b}) \rangle = \langle \text{bit_comp}(\mathbf{a}), \mathbf{b} \rangle$$

Let a be an N dimensional vector and b be a $k = N / \log q$ dimensional vector. Then it holds that

$$\langle \mathbf{a}, \text{powers_of_two}(\mathbf{b}) \rangle = \langle \text{bit_comp}(\mathbf{a}), \mathbf{b} \rangle,$$

because $\sum a_{ij} \cdot (2^j \cdot b_i) = \sum (a_{ij} \cdot 2^j) \cdot b_i$.

EXAMPLE $\langle \text{flatten}(a), \text{powers_of_two}(b) \rangle = \langle \text{bit_comp}(a), b \rangle$

For example, let $a = (2, 3, 0)$, $b = (3,)$, $q = 8$.

1. `bit_comp` on a gives $2 \cdot 2^2 + 3 \cdot 2 + 0 \cdot 1 = 6$
2. `bit_decomp` on 6 gives $(1, 1, 0)$,
3. For the left-hand side we have $6 \cdot 3 \bmod 8 = 2$.
4. For the right-hand side we have $1 \cdot 2^2 \cdot 3 + 1 \cdot 2^1 \cdot 3 + 0 \cdot 2^0 \cdot 3 = 2$.

The same example in Sage:

```
q = 8
R = IntegerModRing(q)
a = vector(R, (2,3,0))
b = vector(R, (3,))

bit_comp(a).dot_product(b) == flatten(a).dot_product(powers_of_two(b))
```

G AND G^-

We can write **bit_comb** operating on vectors of length $n \cdot \lfloor \log q \rfloor$ as multiplication with $\mathbf{G} \in \mathbb{Z}_q^{n \times n \cdot \lfloor \log q \rfloor}$:

$$\mathbf{g} := (1, 2, \dots, 2^{\lfloor \log q \rfloor})$$

$$\mathbf{G} := \mathbf{g}^T \otimes \mathbf{I}_n = \begin{pmatrix} \mathbf{g}^T & \mathbf{0}^T & \dots \\ \mathbf{0}^T & \mathbf{g}^T & \dots \end{pmatrix} = \begin{pmatrix} 1 & 2 & \dots & 2^{\lfloor \log q \rfloor} & 0 & 0 & \dots & 0 & \dots \\ 0 & 0 & \dots & 0 & 1 & 2 & \dots & 2^{\lfloor \log q \rfloor} & \dots \end{pmatrix}$$

The inverse is component-wise **bit_decomp**, motivating the notation G^- for this operation:

$$\mathbf{v} = \mathbf{G} \cdot G^-(\mathbf{v})$$

.

- Multiplication with $\mathbf{G}$ is linear
- The function G^- is highly non-linear

GSW

Craig Gentry, Amit Sahai, and Brent Waters. **Homomorphic Encryption from Learning with Errors: Conceptually-Simpler, Asymptotically-Faster, Attribute-Based**. In: *CRYPTO 2013, Part I*. ed. by Ran Canetti and Juan A. Garay. Vol. 8042. LNCS. Springer, Berlin, Heidelberg, Aug. 2013, pp. 75–92. DOI: [10.1007/978-3-642-40041-4_5](https://doi.org/10.1007/978-3-642-40041-4_5)

APPROXIMATE EIGENVALUES

- In GSW, we work with **approximate** eigenvalue equations:

$$\mathbf{s}^T \cdot \mathbf{A} \approx \lambda \cdot \mathbf{s}^T$$

where $\approx$ means equality up to small error.

- The secret key $\mathbf{s}$ is the approximate eigenvector.
- The message μ (encoded via $\mathbf{G}$) is related to the eigenvalue

Homomorphic operations preserve this structure

- Addition: eigenvalues add
- Multiplication: eigenvalues multiply

Secret key is a vector

$$\mathbf{s} = \begin{pmatrix} \mathbf{t} \\ -1 \end{pmatrix} \quad \text{where } \mathbf{t} \in \mathbb{Z}_q^n$$

Random matrix $\mathbf{A} \in \mathbb{Z}_q^{(n+1) \times m}$ such that

$$\mathbf{s}^T \cdot \mathbf{A} \approx \mathbf{0} \bmod q$$

- Choose random $\mathbf{B}^{n \times m}$ and let

$$\mathbf{A} := \begin{pmatrix} \mathbf{B} \\ \mathbf{t}^T \cdot \mathbf{B} + \mathbf{e}' \end{pmatrix}$$

- Encryption of $\mu \in \{0, 1\}$:

$$\mathbf{C} = \mathbf{A} + \mu \cdot \mathbf{G}$$

- $\mathbf{G}$ is the gadget matrix from earlier.

- Given ciphertext $\mathbf{C}$ and secret key $\mathbf{s}$:

$$\mathbf{s}^T \cdot \mathbf{C} \approx \mu \cdot \mathbf{s}^T \cdot \mathbf{G}$$

- We call this an **approximate eigenvalue equation**.
- $\mathbf{G}$ serves as an error-correcting artefact for μ .
- Decryption recovers μ from $\mathbf{s}^T \cdot \mathbf{C}$ by comparing with $\mathbf{s}^T \cdot \mathbf{G}$

Addition output $C_1 + C_2$

- Result: encryption of $\mu_1 + \mu_2$

Multiplication output $C_1 \cdot G^-(C_2)$

- Result: encryption of $\mu_1 \cdot \mu_2$
- $G^-(\cdot)$ is the **bit decomposition function** which produces small coefficients

Complete

These operations suffice to compute all Boolean functions

TOY SCHEME: EXACT EIGENVALUES

Variant where ciphertext is $\mathbf{C} = \mathbf{A} + \mu \cdot \mathbf{I}$ with $\mathbf{s}^T \cdot \mathbf{A} = \mathbf{0}$ (exact, no error).

- Then $\mathbf{s}^T \cdot \mathbf{C} = \mu \cdot \mathbf{s}^T$ (exact eigenvalue equation)

Addition $\mathbf{C}_1 + \mathbf{C}_2$ eigenvalues add:

$$\mathbf{s}^T \cdot (\mathbf{C}_1 + \mathbf{C}_2) = (\mu_1 + \mu_2) \cdot \mathbf{s}^T$$

Multiplication $\mathbf{C}_1 \cdot \mathbf{C}_2$ eigenvalues multiply:

$$\mathbf{s}^T \cdot (\mathbf{C}_1 \cdot \mathbf{C}_2) = \mu_1 \cdot \mathbf{s}^T \cdot \mathbf{C}_2 = \mu_1 \cdot \mu_2 \cdot \mathbf{s}^T$$

- Real ciphertext: $\mathbf{C} = \mathbf{A} + \mu \cdot \mathbf{G}$ with $\mathbf{s}^T \cdot \mathbf{A} \approx 0$
- Then $\mathbf{s}^T \cdot \mathbf{C} \approx \mu \cdot \mathbf{s}^T \cdot \mathbf{G}$ (approximate eigenvalue equation)
- **Addition:** $\mathbf{C}_1 + \mathbf{C}_2$

$$\mathbf{s}^T \cdot (\mathbf{C}_1 + \mathbf{C}_2) \approx (\mu_1 + \mu_2) \cdot \mathbf{s}^T \cdot \mathbf{G}$$

- Approximate eigenvalues add (if you don't do it too many times)
- Error accumulates linearly with additions

GSW: MULTIPLYING WITH ERRORS

Multiplication: $C_1 \cdot G^-(C_2)$

$$\mathbf{s}^T \cdot (C_1 \cdot G^-(C_2)) = (\mathbf{s}^T \cdot C_1) \cdot G^-(C_2) \approx \mu_1 \cdot \mathbf{s}^T \cdot \mathbf{G} \cdot G^-(C_2) = \mu_1 \cdot (\mathbf{s}^T \cdot C_2) \approx \mu_1 \cdot \mu_2 \cdot \mathbf{s}^T \cdot \mathbf{G}$$

Because $G^-(C_2)$ is small:

$$\begin{aligned} (\mathbf{s}^T \cdot C_1) \cdot G^-(C_2) &= (\mu_1 \cdot \mathbf{s}^T \cdot \mathbf{G} + \mathbf{e}_1) \cdot G^-(C_2) \\ &= \mu_1 \cdot \mathbf{s}^T \cdot \mathbf{G} \cdot G^-(C_2) + \mathbf{e}_1 \cdot G^-(C_2) \approx \mu_1 \cdot \mathbf{s}^T \cdot \mathbf{G} \cdot G^-(C_2) \end{aligned}$$

Because μ_1 is small:

$$\mu_1 \cdot (\mathbf{s}^T \cdot C_2) = \mu_1 \cdot (\mu_2 \cdot \mathbf{s}^T \cdot \mathbf{G} + \mathbf{e}_2) = \mu_1 \cdot \mu_2 \cdot \mathbf{s}^T \cdot \mathbf{G} + \mu_1 \cdot \mathbf{e}_2 \approx \mu_1 \cdot \mu_2 \cdot \mathbf{s}^T \cdot \mathbf{G}$$

Overall

$$\mathbf{s}^T \cdot (C_1 \cdot G^-(C_2)) = \mu_1 \cdot \mu_2 \cdot \mathbf{s}^T \cdot \mathbf{G} + \mu_1 \cdot \mathbf{e}_2 + \mathbf{e}_1 \cdot G^-(C_2)$$

MULTIPLICATIVE DEPTH

$$\mathbf{C}_3 := \mathbf{C}_1 \cdot G^-(\mathbf{C}_2)$$

- Assume $\mathbf{C}_i, \mu_i, \mathbf{e}_i$ are all $\leq B$ in absolute value for $i \in \{0, 1\}$
- After multiplication $\mathbf{C}_3$ is bounded by $m \cdot B^2$
- After L levels: noise grows worse than $B^{(2^L)}$
- Require $B^{(2^L)} < q$, otherwise wrap-around destroys signal
- Also q/B must be sub-exponential in n for security
 - Exponential gap enables LLL attack
- Result: scheme as-is evaluates polynomials of degree sublinear in n

CONTROLLING THE NOISE

Strongly B bounded matrices:

- $\mathbf{C}$ is strongly B bounded if:
 - entries of $\mathbf{C}$ are bounded by 1
 - Error $\mathbf{e}_i$ is bounded by B

NAND:

- Use **NAND** gate: $\mathbf{C}_3 = \mathbf{G} - \mathbf{C}_1 \cdot \mathbf{G}^-(\mathbf{C}_2)$
 - Universal gate: can build any circuit
- Ensures messages μ_i stay $\in \{0, 1\}$
 - Critical: avoids $\mu_2 \cdot e_1$ term blowing up
- If $\mathbf{C}_1, \mathbf{C}_2$ strongly B bounded:
 - Error vector of $\mathbf{C}_3$ bounded by $m \cdot B$ (not B^2)
 - After **flatten**: entries bounded by 1 again
 - $\mathbf{C}_3$ is strongly $m \cdot B$ bounded
- Circuits of depth d give $m^d \cdot B$ noise
- Polynomial **depth** when q/B is sub-exponential

BRAKERSKI-VAIKUNTANATHAN: MULTIPLICATION ORDER

- Order of multiplications matters for noise (example: degree 4, depth 2)
- **Balanced tree** (traditional): $C_{out} = (C_1 \cdot C_2) \cdot (C_3 \cdot C_4)$
 - Noise: $\|e_{out}\|_{\infty} \approx m^2 \cdot B$
 - For ℓ multiplications: noise $\approx \text{poly}(m)^{\log \ell}$
- **Sequential** (left-to-right): $C_{out} = (((C_4 \cdot C_3) \cdot C_2) \cdot C_1)$
 - Noise: $e_{out} \approx 4 m \cdot B$
 - For ℓ multiplications: noise $\approx \ell \cdot \text{poly}(m)$
 - Each 0,1 matrix multiplied by some e_i only
- Sequential evaluation gives **linear** noise growth in number of multiplications

Brakerski-Vaikuntanathan observation

Zvika Brakerski and Vinod Vaikuntanathan. **Lattice-based FHE as secure as PKE**. In: *ITCS 2014*. Ed. by Moni Naor. ACM, Jan. 2014, pp. 1–12. DOI: 10.1145/2554797.2554799

How to evaluate circuits of any polynomial depth?

Answer: Gentry's bootstrapping technique

- Requires **circular security** of GSW encryption
- Cannot currently reduce circular security to LWE

THE BOOTSTRAPPING IDEA

- Suppose we have decryptable ciphertexts C_1, C_2 (noise $< q/4$) but not computable (would become undecryptable after another gate)
- Naive idea: if we had the secret key, decrypt and re-encrypt with fresh noise
 - But then why use FHE at all?
- Better idea: have a ciphertext C' of the secret key encrypted under itself (circular encryption)
 - Homomorphically evaluate the decryption circuit on C'

BootNAND Circuit

- Define the bootstrapping circuit:

$$\text{BootNAND}_{C_1, C_2}(\mathbf{s}) = \text{Dec}_{\mathbf{s}}(C_1) \text{ NAND } \text{Dec}_{\mathbf{s}}(C_2)$$

- Input: $C' = \text{Enc}_{\mathbf{s}}(\mathbf{s})$ (fresh circular encryption of $\mathbf{s}$)
- Output: encryption of $\mu_1 \text{ NAND } \mu_2$ under $\mathbf{s}$.

Key requirements

- **BootNAND** has small depth (small enough to evaluate)
- Output is a fresh encryption with small noise

BOOTSTRAPPING EVERY GATE

- Given ciphertexts C_1, C_2 for two wires:
 1. Construct $\text{BootNAND}_{C_1, C_2}$ circuit
 2. Evaluate homomorphically on $C' = \text{Enc}_s(s)$
- Output: fresh encryption of the NAND result with small noise
- Repeat for every gate: this gives **fully** homomorphic encryption

Plaintext computation:

$$\mu_1, \mu_2 \xrightarrow{\text{NAND}} \mu_3, \mu_4 \xrightarrow{\text{NAND}} \mu_5 \dots$$



Bootstrapped homomorphic computation:

$$C_1, C_2 \xrightarrow{\text{BootNAND}} C_3, C_4 \xrightarrow{\text{BootNAND}} C_5 \dots$$

- Is it OK to publish a circular encryption $\text{Enc}_s(\mathbf{s})$?
- IND-CPA security alone does **not** guarantee safety in this scenario
 - Can construct IND-CPA schemes whose security **breaks completely** with circular encryption
- For Regev encryption: circular security follows from LWE
- For GSW: showing circular security from LWE is an open problem in lattice-based cryptography

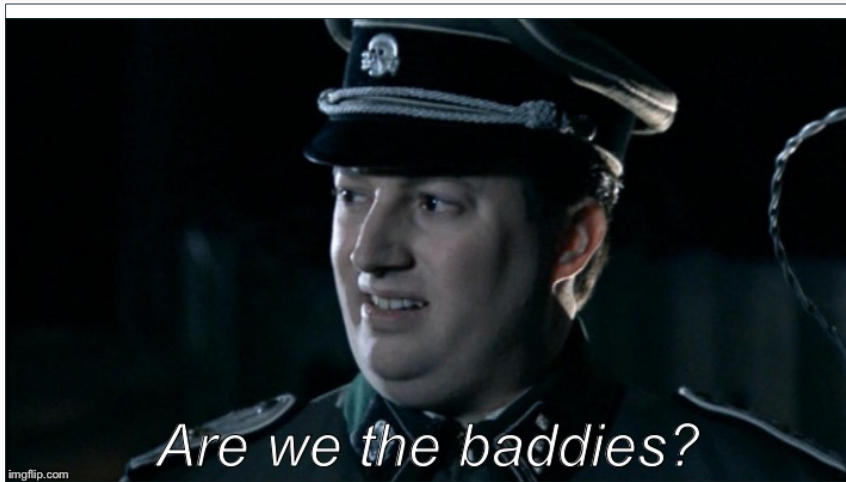
APPLICATIONS

EXAMPLE APPLICATIONS

Recent Real-time FHE Applications

Application	Real-world Use case
<i>Encrypted Facial Recognition</i>	• <i>Fintech:</i> Protects facial templates stored and used in the facial payment solution
<i>Encrypted RAG (Retrieval Augmented Generation)</i>	• <i>Ministry of National Defense:</i> Protects confidential information in the military while enabling augmented generations in LLMs • <i>Personal AI Chatbot in U.S.:</i> Protects personal conversations and information while providing virtual counseling
<i>Encrypted Semantic Search</i>	• <i>Cloud Provider:</i> Protects saved images while providing convenient semantic searches
<i>Encrypted Object Detection</i>	• <i>Army:</i> Protects model weights in the object detection workload
<i>Encrypted Data Analytics and Statistics</i>	• <i>Statistics Bureau:</i> Maintains confidentiality of source data while performing statistical analyses

EXAMPLE APPLICATIONS



PRIVATE INFORMATION RETRIEVAL (PIR)

Goals:

- Read a bit from a database.
- Hide from the database server which bit you read.

Easy extensions:

- Longer DB rows
- Queries by keyword
- Read 10 rows at once

PRIVATE INFORMATION RETRIEVAL: REQUIREMENTS

Correctness For all databases $x \in \{0, 1\}^N$, all queries $i \in [N]$, an honest client interacting with an honest server outputs x_i (whp).

Client privacy The server “learns nothing” about the client’s query.

- For all databases $x \in \{0, 1\}^N$, for all queries $i, j \in [N]$, View of server when client queries $i \approx_c$ View of server when client queries j

Communication efficiency A read over a database of N bits requires $\ll N$ bits of communication.

APPLICATIONS OF PIR

- Private DNS
- Private search
- Anonymous messaging
- Private Netflix

PIR COMPUTATION V COMMUNICATION

- **PIR requires very little communication**
 - DB read with no privacy: $\log N$ bits
 - DB read with PIR: $\approx 256 \log N$ bits [BGI16]
- **PIR requires lots of server-side computation**
 - DB read with no privacy: $\mathcal{O}(1)$ time
 - DB read with PIR: $\geq N$ time $\leftarrow$ Inherent!

Intuition

If server doesn't touch database bit i while answering client's query, client can't be reading bit i .

“SQUARE-ROOT” SINGLE-SERVER PIR [KO97]

$$\begin{pmatrix} \dots & d_1 & \dots & \dots \\ \dots & d_2 & \dots & \dots \\ \dots & \vdots & \dots & \dots \\ \dots & d_{\sqrt{N}} & \dots & \dots \end{pmatrix} \cdot \begin{pmatrix} \text{Enc}(0) \\ \text{Enc}(1) \\ \text{Enc}(0) \\ \text{Enc}(0) \end{pmatrix} = \begin{pmatrix} \text{Enc}(d_1) \\ \text{Enc}(d_2) \\ \vdots \\ \text{Enc}(d_{\sqrt{N}}) \end{pmatrix}$$

- Arrange the N -bit database into a $\sqrt{N} \times \sqrt{N}$ matrix $\mathbf{D}$.
- Client wants column $c \in \{1, \dots, \sqrt{N}\}$
 - Uses linearly homomorphic encryption
- Client sends encryption of standard basis vector $\mathbf{e}_c \in \{0, 1\}^{\sqrt{N}}$
- Server computes matrix-vector product: $\mathbf{D} \cdot \mathbf{e}_c$
- Result is an encryption of the c -th column of $\mathbf{D}$ (which contains $\sqrt{N}$ elements)

Correct and secure via encryption scheme

Comm. efficient $\sqrt{N}$ ciphertexts up and down

Server computation $\Omega(N)$ operations (must multiply by the whole matrix).

“SQUARE-ROOT” SINGLE-SERVER PIR WITH REGEV

- Database is a matrix $\mathbf{D} \in \{0, 1\}^{\sqrt{N} \times \sqrt{N}}$
- Public matrix (Hint) $\mathbf{A} \in \mathbb{Z}_q^{\sqrt{N} \times n}$
- Query (encryption of $\mathbf{e}_c$;) $\mathbf{b} \in \mathbb{Z}_q^{\sqrt{N} \times 1}$
- Server computes $\mathbf{D} \cdot \mathbf{A} \in \mathbb{Z}_q^{\sqrt{N} \times n}$ and $\mathbf{D} \cdot \mathbf{b} \in \mathbb{Z}_q^{\sqrt{N} \times 1}$
- Computation is $(n + 1) \cdot N$ operations in $\mathbb{Z}_q$
 - since $\mathbf{D} \cdot \mathbf{A}$ takes $N \cdot n$ ops and $\mathbf{D} \cdot \mathbf{b}$ takes N ops
- Communication is $2(n + 1) \cdot \sqrt{N}$ elements of $\mathbb{Z}_q$
- For security, we need $n \approx 1000$, so:
 - For each byte of database the server performs 1000 32-bit ops

REFERENCES

- Alexandra Henzinger, Matthew M. Hong, Henry Corrigan-Gibbs, Sarah Meiklejohn, and Vinod Vaikuntanathan. **One Server for the Price of Two: Simple and Fast Single-Server Private Information Retrieval**. In: *USENIX Security 2023*. Ed. by Joseph A. Calandrino and Carmela Troncoso. USENIX Association, Aug. 2023, pp. 3889–3905. URL: <https://www.usenix.org/conference/usenixsecurity23/presentation/henzinger>
- Alex Davidson, Gonalo Pestana, and Sofía Celi. **FrodoPIR: Simple, Scalable, Single-Server Private Information Retrieval**. In: *PoPETs 2023.1* (Jan. 2023), pp. 365–383. DOI: 10.56553/popets-2023-0022

PRIVATE SET INTERSECTION (PSI)

- Two parties: **sender** X and **receiver** Y
- Goal: receiver learns $X \cap Y$, sender learns nothing
- **Unbalanced** PSI: $|X| \gg |Y|$ (e.g. server vs. mobile device)

Applications

- Private contact discovery (messaging apps)
- Password leak validation

PRIVATE SET INTERSECTION (PSI) FROM FHE

Polynomial trick

For a set $S = \{x_1, \dots, x_B\}$, define

$$P_S(y) = r \cdot \prod_{i=1}^B (y - x_i)$$

$\Downarrow$

$$P_S(y) = 0 \iff y \in S \text{ (for } r \in \mathbb{F}^*)$$

- Receiver encrypts y and sends $\text{Enc}(\text{pk}, y)$ to sender
- Sender homomorphically evaluates $P_S(y)$ to get $\text{Enc}(P_S(y))$
- Receiver decrypts: result is zero iff $y \in X$

Hao Chen, Zhicong Huang, Kim Laine, and Peter Rindal. **Labeled PSI from Fully Homomorphic Encryption with Malicious Security**. In: *ACM CCS 2018*. Ed. by David Lie, Mohammad Mannan, Michael Backes, and XiaoFeng Wang. ACM Press, Oct. 2018, pp. 1223–1237. DOI: 10.1145/3243734.3243836

FHE ciphertexts are typically **smaller** than the messages exchanged in the schemes discussed in the previous lecture.

- Simple idea:
 1. Client FHE encrypts x as $[x]$
 2. Server homomorphically computes PRF using plaintext k and $[x]$ to obtain $[F_k(x)]$
 3. Client FHE decrypts $F_k(x)$
- Problem: PRFs need deep circuits, deep circuits are expensive
- Proposal: Use Dark Matter (weak-)PRF candidate [BIPSW18] $\sum (\mathbf{A} \cdot \mathbf{x} \bmod 2) \bmod 3$ where $\mathbf{A}$ is the secret key
- This can be computed with one level of FHE bootstrapping